

클라우드 인프라 교육 · 2일차 오전

클라우드 컴퓨팅 기술

퍼블릭 클라우드(AWS) 서비스 구성·관리 실습

실습 0 AWS 콘솔 접속과 기본 설정

실습 1 EC2 인스턴스 생성

실습 2 EC2 접속과 리눅스 기초

실습 3 S3 버킷과 객체 관리

실습 4 VPC·보안그룹 구조 확인

오늘 오전의 목표

- **첫 서버 만들기** — EC2 인스턴스를 생성·접속·종료하는 전 과정을 직접 수행한다.
- **저장소 다루기** — S3 버킷을 만들고 파일을 올리며 접근 권한의 동작을 확인한다.
- **네트워크 읽기** — 어제 배운 VPC·서브넷·보안그룹이 실제 콘솔에서 어떻게 생겼는지 확인한다.
- **리소스 정리** — 사용이 끝난 리소스는 항상 정리한다.

시작 전 — 비용 주의

**실습은 강사가 제공하는 공용 계정의 IAM 사용자로 진행한다.
세션이 끝나면 [정리하기] 단계를 절대 건너뛰지 않는다.**

- 배포받은 계정(student##)과 공통 비밀번호로 로그인
- 실습 정책상 t2/t3.micro만 생성 가능, 서울 리전만 사용 가능하도록 제한되어 있음
- EC2는 켜져 있는 동안 과금 — 세션 끝에 반드시 중지

실습 0

AWS 콘솔 접속과 기본 설정

로그인 · 리전 설정 · 콘솔 사용법

콘솔 접속

1

로그인 URL 접속 (<https://signin.aws.amazon.com/console>)

2

배포받은 IAM 사용자 이름(student##)과 공통 비밀번호 입력

3

로그인 후 우측 상단 리전을 [아시아 태평양(서울) ap-northeast-2]로 변경

4

상단 검색창에 EC2를 입력해 검색 이동을 연습

리전과 가용영역 — 실습 관점 복습

- **리전 확인 습관** — 콘솔 우측 상단이 항상 "서울"인지 확인. 다른 리전에 만든 자원은 안 보여서 "사라졌다"고 착각하기 쉽다
- **AZ는 어디서 보나** — EC2 생성 시 서브넷 선택란, 인스턴스 상세의 가용영역 항목
- **요금 주의** — 리전마다 단가가 다르다. 실습은 서울 고정

콘솔 사용 팁

- **검색창이 최고의 내비게이션** — 메뉴를 헤매지 말고 서비스 이름을 바로 검색
- **새 탭 활용** — EC2와 VPC를 각각 탭으로 열어 두면 오가기 편함
- **새로고침** — 상태 변화(생성 중→실행 중)는 자동 갱신이 늦을 수 있음. 목록의 새로고침 버튼 활용
- **이름 규칙** — 오늘 만드는 모든 자원에 본인 이니셜을 붙인다 (예: web-hgd). 공용 계정에서 내 것을 구분하는 방법

실습 1

EC2 인스턴스 생성

나의 첫 클라우드 서버 — 7단계

[Day1 복습] EC2를 만들기 전에 — 개념 되살리기

- **EC2의 정체** — Day1 오후 가상화 그림 그대로: 물리 서버 → 하이퍼바이저 → 그 위의 VM 하나가 지금 만들 인스턴스다
- **AMI** — "VM은 파일이다"에서 나온 템플릿 개념. OS+구성이 담긴 복제 원본
- **타입 이름 규칙** — t3.micro = t(버스트형 범용) + 3(세대) + micro(크기)
- **과금 기억** — 실행 중=컴퓨팅+EBS 과금 / 중지=EBS만 과금 / 종료=삭제. 오늘 종일 이 규칙대로 관리한다

EC2 생성 — 전체 7단계 미리보기

1

이름 입력 → ② AMI(운영체제) 선택 → ③ 인스턴스 타입 선택

2

④ 키 페어 생성 → ⑤ 네트워크·보안그룹 설정

3

⑥ 스토리지 확인 → ⑦ 시작 및 상태 확인

4

각 단계에서 "왜 이 선택인지"를 함께 이해하는 것이 목표

단계 ① 이름 / ② AMI 선택

- **이름** — my-first-server-이니셜 (예: my-first-server-hgd)
- **AMI란** — 서버에 설치될 OS와 초기 구성이 담긴 디스크 이미지 (어제 배운 VM 템플릿 개념의 AWS 구현)
- **선택** — Amazon Linux 2023. AWS 최적화 무료 리눅스
- **참고** — Ubuntu·Windows 등도 선택 가능. OS에 따라 접속 계정명이 달라짐
 - Amazon Linux - ec2-user
 - Ubuntu - ubuntu
 - CentOS - centos or ec2-user
 - Debian - admin
 - Fedora - fedora or ec2-user
 - RHEL(Red Hat) - ec2-user or root
 - SUSE - ec2-user or root

단계 ③ 인스턴스 타입

- **선택** — t2.micro 또는 t3.micro (1~2 vCPU, 1GB) — 실습 계정 정책상 이 타입만 생성 허용
- **어제 배운 규칙** — t(버스트형 범용) + 세대 + micro(크기)
- **버스트형 특성** — 평시 저성능, 순간적으로 성능 상승. 실습·저부하 웹에 적합
- **주의** — 큰 타입(xlarge 등)을 선택하면 정책에 의해 생성이 거부된다(권한 오류) — micro를 선택할 것

EC2 인스턴스 패밀리

패밀리	예시	특성	주 용도
범용 (T/M)	t3.micro, m5.large	CPU-메모리 균형, T는 버스트형	웹 서버, 개발-테스트
컴퓨팅 최적화 (C)	c5.xlarge	vCPU 비중 높음	고성능 연산, 게임 서버
메모리 최적화 (R/X)	r5.xlarge	메모리 비중 높음	인메모리 캐시, 대용량 DB
스토리지 최적화 (I/D)	i3.large	고속 로컬 디스크	NoSQL, DW
가속 컴퓨팅 (P/G)	p4d, g5	GPU 탑재	AI 학습추론, 렌더링

접미사	의미	설명
a	AMD CPU	인텔 대신 AMD EPYC 프로세서. 동급 인텔 대비 약 10% 저렴
g	Graviton (ARM)	AWS 자체 개발 ARM 칩. 가성비 최고, 단 ARM용 소프트웨어 필요
i	Intel	인텔임을 명시 (최신 세대에서 구분용)
d	로컬 디스크 (Instance Store)	서버 본체에 NVMe SSD 직결. 초고속이지만 인스턴스 중지·종료 시 데이터 소멸 (EBS와 다름!)
n	네트워크 강화	네트워크 대역폭 대폭 상향 (최대 100Gbps급)
e	확장 (extra)	메모리 또는 스토리지 용량 확대
z	고클럭	CPU 클럭 속도 극대화 (라이선스가 코어당 과금인 DB 등)
접미사 없음	기본형	인텔 CPU + EBS 스토리지 + 표준 네트워크

단계 ④ 키 페어 — 서버의 열쇠

- **키 페어란** — 서버 접속용 공개키/개인키 쌍. 비밀번호 대신 키 파일로 인증
- **생성** — [새 키 페어 생성] → 이름 my-key-이니셜 → RSA / .pem 선택
- **중요** — 다운로드되는 .pem 파일은 다시 받을 수 없다. 분실 시 접속 불가 → 안전하게 보관
- **보안 원칙** — 키 파일을 메신저·메일로 공유하지 않는다

단계 ⑤-1 네트워크 설정 — 화면의 항목들 읽기

- **VPC** — 기본 VPC 유지. 계정마다 리전별로 하나씩 미리 만들어져 있는
 - **서브넷** — 서브넷 선택이 곧 AZ 선택이다. 서브넷은 AZ 하나에 고정되므로(Day1), EC2 화면에 AZ 칸이 따로 없다. "기본 설정 없음"이면 자동 배정
 - **퍼블릭 IP 자동 할당** — [활성화] 확인. 이게 꺼져 있으면 서버는 생성되지만 인터넷에서 접속할 주소가 없다
 - **방화벽(보안 그룹)** — [보안 그룹 생성] 선택. 이름: sg_ssh_이니셜.
-
- 리전 : 서울시
 - 가용 영역(AZ): 강남구, 강북구 (물리적으로 떨어진 동네)
 - 서브넷: 강남구에 있는 아파트 단지 (가용 영역 내부에 생성하는 네트워크 공간)
 - 서브넷을 선택하는 것이 곧 어느 물리 데이터 센터(가용 영역)에 내 서버를 둘지 결정하는 것

단계 ⑤-2 보안그룹의 원리

- **무엇인가** — 인스턴스 단위로 붙는 가상 방화벽. 서버로 들어오고 나가는 모든 데이터는 이 규칙을 통과해야 한다
- **인바운드 / 아웃바운드** — 인바운드는 외부에서 내 서버로, 아웃바운드는 내 서버에서 외부로
- **인바운드** — 인바운드는 기본적으로 '모두 잠김' 상태, ex) 22 포트 열어줘, 80번 포트 열어줘라고 허용 목록을 작성
- **아웃바운드** — 아웃바운드는 기본적으로 '모두' 열림
- **Stateful** — 내가 허용한 요청에 대한 응답은 자동으로 통과. "나간 요청의 답까지 규칙을 쓸 필요 없다"는 뜻
- **허용만 가능** — 보안그룹에는 거부(Deny) 규칙이 없다. "누구를 막을까"가 아니라 "누구만 들일까"로 사고한다

단계 ⑤-3 규칙 한 줄의 4요소 — 오늘 입력할 값

요소	의미	오늘 입력값	비고
유형	미리 정의된 서비스 이름	SSH	고르면 프로토콜·포트 자동 입력
프로토콜	통신 규약	TCP	SSH는 TCP 기반
포트 범위	열어 줄 문 번호	22	SSH 표준 포트
소스	누구에게 여는가	[내 IP] 선택 → x.x.x.x/32	가장 중요한 칸 — 다음 슬라이드
소스	누구에게 여는가	13.209.1.56/29 입력	AWS 웹 콘솔 접속용(중계 서버)

읽는 법: "TCP 22번 문을, 이 IP 한 명에게만 연다" — 규칙 하나가 곧 하나의 문장이다

보안그룹을 그림으로 — 문마다 명단이 다르다



22번 문은 나에게만, 80번 문은 모두에게(웹 공개 시), 나머지는 아예 닫힘 — 문마다 다른 출입 명단

단계 ⑤-4 왜 22번은 반드시 [내 IP]인가

- **현실** — 퍼블릭 IP를 받는 순간, 전 세계의 자동 스캐너들이 수 분 안에 내 서버의 열린 포트를 두드리기 시작한다. 과장이 아니라 인터넷의 평상시 상태다
- **22를 전체 개방하면** — 무차별 비밀번호 대입(브루트포스) 시도가 즉시 유입. 로그가 실패 기록으로 도배되고, 키가 약하거나 유출되면 서버 장악 → 코인 채굴·공격 경유지로 악용
- **[내 IP]의 의미** — 소스가 x.x.x.x/32(딱 나 하나)로 입력됨 → 나 외의 접속 시도는 문 앞에서 차단. 공격 표면이 사실상 0
- **예외는 웹 포트뿐** — 80/443은 불특정 다수가 봐야 하니 0.0.0.0/0이 정상. "관리 통로는 좁게, 서비스 통로는 목적만큼"
- **주의** — 내 IP는 바뀔 수 있다(장소 이동, 공유기 재시작). 갑자기 접속이 안 되면 보안그룹의 내 IP를 현재 IP로 갱신하는 것이 1순위 점검

단계 ⑥ 스토리지 / ⑦ 시작

- **스토리지** — 기본 8GB gp3(범용 SSD) 유지. 어제 배운 EBS 볼륨이 바로 이것
- **요약 확인** — 우측 요약 패널에서 타입(t2/t3.micro)·키 페어·보안그룹 재확인
- **시작** — [인스턴스 시작] 클릭 → 1~2분 후 상태 [실행 중(running)]
- **확인 항목** — 인스턴스 상세에서 퍼블릭 IPv4 주소, 가용영역, 보안그룹을 눈으로 확인

퍼블릭 IP vs 프라이빗 IP

- **퍼블릭 IP** — 인터넷에서 접근하는 주소. 중지 후 재시작하면 바뀔 수 있음
- **프라이빗 IP** — VPC 내부 통신용 주소. 유지됨
- **고정이 필요하다면** — Elastic IP를 할당 (단, 미사용 상태로 방치 시 과금 — 어제 배운 숨은 과금 항목)
- **실습에서는** — 매번 현재 퍼블릭 IP를 확인해 사용

실습 2

EC2 접속과 리눅스 기초

EC2 Instance Connect · 기본 명령어

접속 방법 두 가지

방법	내용	오늘의 선택
EC2 Instance Connect	브라우저에서 바로 터미널 열기. 키 파일 불필요	간단·안전
SSH 클라이언트(PuTTY)	내 PC 터미널에서 .pem 키로 접속	실습 2-B에서 상세 안내

Instance Connect로 접속

1

인스턴스 목록에서 my-first-server 선택 → 상단 [연결]

2

[EC2 인스턴스 연결] 탭에서 [연결] 클릭

3

검은 터미널 화면이 열리면 접속 성공

4

프롬프트에 ec2-user@... 표시 확인

(참고) SSH로 접속하는 경우

```
# 내 PC 터미널에서 (키 파일이 있는 폴더)
chmod 400 my-key.pem          # 키 파일 권한 제한 (필수)
ssh -i my-key.pem ec2-user@퍼블릭IP

# 접속이 안 되면 점검 순서
# 1) 보안그룹 22번 포트가 내 IP에 열려 있는가
# 2) 퍼블릭 IP가 최신인가 (재시작 시 변경)
# 3) 키 파일 경로·권한이 맞는가
```

실무에서는 SSH가 표준 — 오늘은 개념만 알아 두고 Instance Connect로 진행

왜 PuTTY인가

- **Instance Connect의 한계** — 브라우저 접속은 간편하지만, 세션이 잘 끊기고 복사·붙여넣기가 불편.
- **PuTTY** — Windows에서 가장 널리 쓰이는 무료 SSH 클라이언트. 실무 현장의 표준 도구 중 하나
- **참고** — Windows 10 이후는 명령 프롬프트의 ssh 명령으로도 가능(마지막 슬라이드). PuTTY는 GUI·세션 저장이 장점

시작 전 준비물 체크

항목	확인 방법	비고
실행 중인 EC2	콘솔에서 상태 running + 검사 2/2	중지 상태면 [시작]
퍼블릭 IPv4 주소	인스턴스 상세에서 복사	중지→시작 시 바뀜!
키 페어 파일 (.pem)	EC2 생성 때 다운로드한 my-key.pem	분실 시 접속 불가
보안그룹 SSH(22)	인바운드에 22 ← 내 IP 존재	IP 바뀌면 갱신

네 가지 중 하나라도 빠지면 접속 실패 — 문제의 90%는 이 표에서 해결된다

1단계 — PuTTY 설치

1

공식 사이트 putty.org 접속 → Download 링크 클릭

2

Windows 64-bit MSI 설치 파일(putty-64bit-x.xx-installer.msi) 다운로드

3

설치 실행 — 기본 옵션 그대로 [Next] 연타로 완료

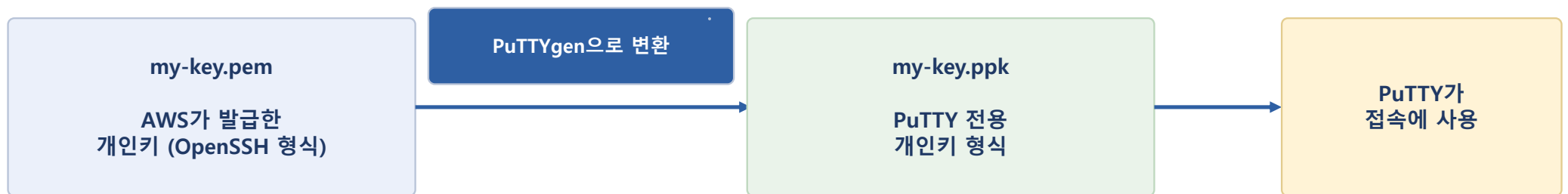
4

설치 확인: 시작 메뉴에 PuTTY와 PuTTYgen 두 프로그램이 생김

5

PuTTYgen이 함께 설치되는 이유 — 다음 슬라이드의 키 변환에 필요

핵심 개념 — .pem과 .ppk는 형식이 다르다



PuTTY는 .pem을 직접 읽지 못한다 — 접속 전 딱 한 번, PuTTYgen으로 .ppk로 변환하는 과정이 필수

2단계 — PuTTYgen으로 키 변환 (.pem → .ppk)

- 1 시작 메뉴에서 PuTTYgen 실행
- 2 [Load] 클릭 → 파일 선택창 우측 하단을 [All Files (*.*)]로 변경 (기본값이면 .pem이 안 보임!)
- 3 my-key.pem 선택 → "Successfully imported foreign key" 메시지 확인 → [확인]
- 4 [Save private key] 클릭 → 암호문구(passphrase) 없이 저장 여부 묻는 창에서 [예] (교육용은 생략, 실무는 설정 권장)
- 5 이름 my-key.ppk로 저장 — 위치를 기억해 둘 것 (예: 바탕화면 키 폴더)

3단계 — PuTTY 세션 설정

1

PuTTY 실행 → 첫 화면(Session 카테고리)

2

Host Name 칸에 입력: ec2-user@퍼블릭IP (예: ec2-user@3.35.10.20)

3

Port: 22 / Connection type: SSH 확인 (기본값 그대로)

4

좌측 트리에서 Connection → SSH → Auth → Credentials 로 이동

5

"Private key file for authentication"의 [Browse...] → 아까 만든 my-key.ppk 선택

4단계 — 세션 저장과 접속

1

좌측 트리 맨 위 Session으로 돌아오기

2

Saved Sessions 칸에 이름 입력 (예: edu-ec2) → [Save] — 다음부터 더블클릭으로 즉시 접속

3

[Open] 클릭 → 검은 터미널 창이 열림

4

최초 접속 시 "PuTTY Security Alert" (서버 호스트키 경고) → [Accept] — 처음 만나는 서버를 기억하겠냐는 확인이며 정상 절차

5

프롬프트에 [ec2-user@ip-xxx ~]\$ 가 보이면 접속 성공!

PuTTY 사용 팁 — 알아두면 편한 것들

- **복사** — 터미널에서 마우스로 드래그하면 자동 복사 (Ctrl+C 아님! Ctrl+C는 명령 중단)
- **붙여넣기** — 터미널에서 마우스 우클릭 = 붙여넣기. 긴 명령은 복사해서 우클릭
- **세션 유지** — Connection 카테고리 → Seconds between keepalives를 30으로 → 방치 시 끊김 방지
- **글자 크기** — Window → Appearance → Font로 조절 (기본이 작아서 강의실에선 14pt 이상 권장)
- **한글 깨짐** — Window → Translation → Remote character set을 UTF-8로

(참고) PuTTY 없이 — Windows 기본 ssh 명령

```
# Windows 10 이후는 명령 프롬프트/PowerShell에 ssh 내장
# 변환(.ppk) 불필요 - .pem을 그대로 사용

cd 키파일있는폴더
ssh -i my-key.pem ec2-user@퍼블릭IP

# "UNPROTECTED PRIVATE KEY" 경고가 나면 (권한 문제):
#   파일 우클릭 > 속성 > 보안 > 고급에서
#   상속 제거 후 본인 계정만 읽기 권한으로 설정
```

도구는 달라도 원리는 동일: "키 파일 + 사용자명@IP + 22번 포트" — 이 세 가지가 SSH 접속의 전부

리눅스 기초 명령 ① — 내가 어디에 있는가

```
whoami      # 현재 사용자 확인 → ec2-user
pwd         # 현재 위치(폴더 경로) 확인
ls          # 현재 폴더의 파일 목록
ls -al      # 숨김 파일·권한까지 자세히
cd /tmp     # 폴더 이동
cd ~        # 홈 폴더로 복귀
```

뒤는 설명 — 입력하지 않아도 된다. 명령만 치고 Enter

리눅스 기초 명령 ② — 서버 상태 읽기

```
cat /etc/os-release    # OS 종류·버전 확인
free -h               # 메모리 사용량 (-h: 읽기 쉬운 단위)
df -h                 # 디스크 사용량
top                   # 실시간 프로세스·CPU (q로 종료)
sudo dnf update -y    # 패키지 업데이트 (sudo=관리자 권한)
```

오늘 오후 Docker 설치에서 sudo dnf 형태를 계속 사용한다

리눅스 기초 명령 ③ — 파일 다루기

```
mkdir work          # 폴더 생성
cd work             # 이동
touch memo.txt      # 빈 파일 생성
echo "hello" > memo.txt # 파일에 내용 쓰기 (덮어쓰기)
cat memo.txt        # 파일 내용 출력
cp memo.txt memo2.txt # 복사
mv memo2.txt note.txt # 이름 변경(이동)
rm note.txt         # 삭제
```

오후 Dockerfile 작성에서 mkdir·cd·cat을 그대로 사용한다

리눅스 기초 명령 ④ — 네트워크 확인

```
curl https://checkip.amazonaws.com # 내 서버의 공인 IP 확인
curl -I https://www.google.com      # 웹 응답 헤더만 확인 (연결 테스트)
ping -c 3 8.8.8.8                   # 네트워크 도달 확인 (3회)
ss -tlnp                             # 열려 있는 포트 목록
```

curl은 오후 실습에서 웹 서버 동작 확인에 계속 사용

중지 vs 종료 — 반드시 구분

- **중지(Stop)** — 서버를 끄되 디스크는 보존. 컴퓨팅 요금 중단, EBS 요금 지속. 재시작 가능
- **종료(Terminate)** — 완전 삭제. 기본 설정에서 루트 볼륨도 함께 삭제. 복구 불가
- **오늘의 운영** — 오전 종료 시 [중지] (오후 Docker 실습에서 재사용) / 3일차 마지막에 [종료]
- **사고 예방** — 종료 버튼은 항상 한 번 더 생각하고 누른다

[Day1 복습] S3 — 어제 배운 것 되살리기

- **오브젝트 스토리지** — 디스크처럼 장착(EBS)하는 게 아니라, 어디서든 HTTP API로 객체를 넣고 꺼내는 저장소
- **EBS와의 차이** — EBS=한 서버의 디스크 / S3=무제한 공용 창고. 스토리지 3유형 그림 기억
- **과금 3요소** — 저장 용량 + 요청 수 + 반출(아웃바운드) 트래픽
- **클래스** — Standard부터 Glacier까지 접근 빈도별 단가 차등 — 오늘 객체 상세에서 눈으로 확인

실습 3

S3 버킷과 객체 관리

버킷 생성 · 업로드 · 접근 권한

S3 실습 전 개념 정리

- **버킷(Bucket)** — 객체를 담는 최상위 컨테이너. 이름은 전 세계에서 유일해야 함
- **객체(Object)** — 저장되는 파일 + 메타데이터. 키(경로+이름)로 식별
- **이름 규칙** — 소문자·숫자·하이픈, 3~63자. 오늘: edu-s3-이니셜-날짜 (예: edu-s3-hgd-0716)
- **요금 구조** — 저장 용량 + 요청 수 + 반출 트래픽 (어제 복습)

버킷 생성과 업로드

1

검색창 S3 → [버킷 만들기]

2

이름: edu-s3-이니셜-날짜 / 리전: 서울 확인

3

[모든 퍼블릭 액세스 차단] 체크 유지 — 기본 안전장치

4

[버킷 만들기] → 생성된 버킷 클릭

5

[업로드] → 이미지나 텍스트 파일 1개 업로드

Access Denied가 뜨는 것이 정상이다

업로드한 객체의 URL을 새 탭에서 열면 → Access Denied

- 퍼블릭 차단이 동작해 아무나 내 파일을 못 본다는 뜻 — 성공 상태
- 꼭 필요한 객체만, 필요한 대상에게만, 서명된 URL 등으로 제한 공개
- 실제 유출 사고 대부분이 이 차단을 함부로 푼 데서 시작

S3 화면에서 함께 볼 것들

- **스토리지 클래스** — 객체 상세에서 Standard 확인 (어제 배운 클래스 체계)
- **버전 관리** — 버킷 속성에서 켤 수 있음. 덮어쓰기·삭제 실수 방어
- **수명주기 규칙** — 오래된 객체를 저가 클래스로 자동 이동 설정 위치 확인
- **폴더 개념** — S3는 폴더라는 개념이 없고 그냥 하나의 파일명. 콘솔이 폴더처럼 보여줄 뿐

실습 4

VPC와 보안그룹 구조 확인

어제 배운 네트워크 개념을 눈으로 확인

[Day1 복습] VPC — 어제 그림을 확인

- **VPC** — SDN이 물리망 위에 소프트웨어로 그려 준 나만의 격리 네트워크. 케이블은 하나도 안 바뀐다
- **구성 요소 5종** — CIDR(IP 대역) / 서브넷(AZ에 고정된 구획) / IGW(인터넷 문) / NAT(나가기 전용) / 라우팅 테이블(길 안내)
- **퍼블릭 서브넷의 정의** — "라우팅에 IGW로 가는 길이 있는 서브넷"
- **보안그룹** — 인스턴스 단위·허용만·Stateful. 아까 만든 규칙이 이 개념의 실물

기본 VPC 구조 확인

1

검색창 VPC → 좌측 [VPC] 메뉴 → 기본 VPC의 CIDR(예: 172.31.0.0/16) 확인

2

좌측 [서브넷] → AZ별로 서브넷이 나뉜 구조 확인 (서브넷 1개 = AZ 1개)

3

좌측 [라우팅 테이블] → 0.0.0.0/0 → igw-... 경로 확인 (인터넷으로 나가는 길)

4

좌측 [인터넷 게이트웨이] → 기본 VPC에 연결된 IGW 확인

[예제] 보안그룹 설계 시나리오 5제

상황	인바운드 규칙	이유
웹사이트를 전 세계에 공개	HTTP(80) ← 0.0.0.0/0	불특정 다수가 봐야 하므로
HTTPS 사이트 공개	HTTPS(443) ← 0.0.0.0/0	암호화 웹 표준 포트
서버 관리는 나만	SSH(22) ← 내 IP/32	관리 통로는 최소 개방
사내에서만 쓰는 관리자 페이지	8080 ← 회사IP대역/24	대역 단위 허용
DB는 웹 서버에서만	MySQL(3306) ← 웹 서버의 보안그룹 ID	IP 대신 SG를 소스로 — 서버가 늘어도 규칙 불변

마지막 예제가 실무 핵심 패턴: 소스에 IP가 아니라 "보안그룹"을 지정할 수 있다

[심화] 소스에 보안그룹을 넣는다는 것

- **문제** — 웹 서버가 Auto Scaling으로 2대→10대가 되면, DB 보안그룹에 IP 10개를 계속 갱신해야 하나?
- **해법** — DB 인바운드 소스를 "웹 서버들이 쓰는 보안그룹 ID(sg-xxxx)"로 지정
- **효과** — 그 보안그룹을 단 서버는 몇 대가 되든 자동으로 허용 대상 — IP 관리 소멸
- **개념 연결** — "IP 기반"이 아니라 "역할(그룹) 기반" 통제 — 제로 트러스트의 신원 기반 사고와 같은 결

보안그룹 실험

1

EC2 인스턴스 상세 → [보안] 탭 → 보안그룹 클릭

2

인바운드 규칙: SSH(22) — 내 IP만 열려 있음을 확인

3

(실험) HTTP(80) — 0.0.0.0/0 규칙을 추가해 본다

4

규칙이 즉시 적용되는 것(재부팅 불필요)을 확인 — Stateful 방화벽

5

실험 후 80 규칙 삭제 (오후 Docker 실습에서 다시 연다)

이 화면들이 말해 주는 것

- **SDN의 실체** — 방금 본 VPC·서브넷·라우팅이 전부 소프트웨어 정의 — 물리 장비를 만진 적이 없다
- **퍼블릭 서브넷의 조건** — IGW로 가는 라우트가 있는 서브넷 (그래서 우리 EC2가 인터넷에서 접근 가능)
- **보안그룹=인스턴스 단위** — 서버마다 다른 규칙 적용 가능
- **최소 개방의 실감** — 22를 내 IP로 좁힌 것과 0.0.0.0/0의 차이를 직접 봄

실습 4-B

나만의 VPC 직접 구축하기

VPC → 퍼블릭 서브넷 → IGW → 라우팅 → 웹 서버 외부 공개까지

오늘 만들 목표 구성도

이 그림을 지금부터 손으로 하나씩 만든다



완성 기준: 내 휴대폰에서도 <http://퍼블릭IP> 로 내 웹페이지 열기

단계 ① VPC 만들기

1

VPC 콘솔 → [VPC 생성] → [VPC만] 선택

2

이름: my-vpc-이니셜 / IPv4 CIDR: 10.0.0.0/16

3

[VPC 생성] → 목록에서 내 VPC 확인

4

(확인) 이 시점의 VPC는 빈 껍데기 — 서브넷도, 인터넷 연결도 아직 없다

단계 ② 서브넷 두 개 만들기

1

[서브넷] → [서브넷 생성] → VPC: my-vpc-이니셜 선택

2

첫 번째 — 이름: public-subnet-이니셜 / AZ: ap-northeast-2a / CIDR: 10.0.1.0/24

3

두 번째 — 이름: private-subnet-이니셜 / AZ: ap-northeast-2c / CIDR: 10.0.2.0/24

4

(확인) 이름은 퍼블릭/프라이빗이지만 아직 둘은 완전히 동일한 상태 — 차이는 다음 단계의 라우팅이 만든다

단계 ③ 인터넷 게이트웨이(IGW)

1

[인터넷 게이트웨이] → [생성] → 이름: my-igw-이니셜

2

생성 직후 상태: Detached (어느 VPC에도 안 붙은 상태)

3

[작업] → [VPC에 연결] → my-vpc-이니셜 선택

4

(확인) 상태가 Attached로 — 이제 VPC에 인터넷으로 나가는 "문"이 생겼다. 하지만 아직 문까지 가는 "길"이 없다

단계 ④ 라우팅 테이블 — 길 만들기

- 1 [라우팅 테이블] → [생성] → 이름: public-rt-이니셜 / VPC: my-vpc 선택 → 생성
- 2 [라우팅] 탭 → [라우팅 편집] → [라우팅 추가]: 대상 0.0.0.0/0 → 타겟 [인터넷 게이트웨이] my-igw 선택 → 저장
- 3 [서브넷 연결] 탭 → [서브넷 연결 편집] → public-subnet만 체크 → 저장
- 4 (핵심) 이 순간 public-subnet이 진짜 퍼블릭 서브넷이 됐다 — "IGW로 가는 라우트가 있는 서브넷"이라는 정의 그대로
- 5 private-subnet은 건드리지 않는다 — 기본 라우팅(local만)이라 인터넷 경로 없음

단계 ⑤ 웹 서버 EC2

```
# root 계정의 패스워드 설정
# sudo passwd
# root 계정 새로운 패스워드 입력
# su

dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>My VPC Web Server - 본인이름</h1>" > /var/www/html/index.html
```

단계 ⑥ 외부 접속 확인 — 완성의 순간

1

인스턴스 실행 중 + 상태 검사 2/2 대기 (2~3분)

2

퍼블릭 IPv4 주소 복사 → 브라우저에서 http://퍼블릭IP (https 아님 주의!)

3

"My VPC Web Server - 본인이름" 페이지 확인

4

진짜 외부 검증: 휴대폰을 와이파이 끄고 LTE로 같은 주소 접속 — 열리면 전 세계 공개 성공

5

이 페이지가 열리기까지 통과한 것들: 인터넷 → IGW → 라우팅 → 서브넷 → 보안그룹 80 → EC2 → httpd

단계 ⑦ 대조 실험 — 프라이빗 서브넷은 정말 막히나

1

같은 설정으로 EC2 1대를 더 생성하되 서브넷만 private-subnet 선택

2

[퍼블릭 IP 자동 할당]을 활성화해도 → 생성 후 보면 퍼블릭 IP가 있어도 접속 불가이거나, 애초에 의미가 없음

3

브라우저 접속 시도 → 응답 없음 (타임아웃)

4

이유: 라우팅 테이블에 IGW로 가는 길이 없다 — 보안그룹이 아니라 "길"의 문제

5

(정리) 실험용 프라이빗 EC2는 확인 후 바로 종료(Terminate)

접속이 안 될 때 — 어디가 문제인지 갈라내는 법

1. 인스턴스	running + 2/2 검사?	아니면 → EC2 자체 문제
2. 퍼블릭 IP	IP가 부여되어 있나?	아니면 → 자동 할당 미체크
3. 라우팅	서브넷 RT에 0.0.0.0/0→IGW?	아니면 → 길이 없음 (프라이빗)
4. 보안그룹	80이 0.0.0.0/0에 열렸나?	아니면 → 문이 잠김
5. 웹 서버	설치 및 권한 설정(root 계정으로 작업)	아니면 → 서버 안 문제